

Semantic Data Normal Forms: Extending Normalization Theory to Vector Embedding Spaces

Anonymous authors

Paper under double-blind review

Abstract

AI-native data systems encode schema meaning with vector embeddings, creating integrity challenges: stochastic instability, semantic drift, and context mixing. We introduce **Semantic Data Normal Forms (SDNF)**, a framework extending classical normalization theory to embedding spaces via a formal *Semantic Relational Schema (SRS)* model and seven normal forms that enforce stability, alias resolution, context isolation, drift bounds, evidence completeness, role consistency, and partition orthogonality. Under explicit assumptions, we derive a conditional cross-context safety guarantee and prove that iterative SDNF governance leads to a stable canonical schema. We implement a compact evidence-driven governance pipeline for SDNF. In a case study on payments data integration, SDNF achieved a 38.7% reduction in schema size (80 attributes consolidated to 49) with $\sim 95\%$ alias merge precision and only $\sim 2\%$ cross-context semantic leakage under our evaluated settings. SDNF thus provides an auditable approach to improve schema reliability in embedding-based data pipelines; extension to other domains is left to future work.

1 Introduction

Modern data schemas act as semantic interfaces between AI models, services, and databases. Unlike symbolic schemas, AI-native systems use vector embeddings to represent dataset attributes; semantic similarity between fields is determined by high-dimensional proximity. This shift introduces three key failure modes: *Stochastic Instability* (repeated encoding of the same concept yields non-identical vectors due to model non-determinism), *Semantic Drift* (model updates or data evolution shift concept representations over time), and *Context Mixing* (distinct domains overlap in embedding space, e.g., “card” meaning a payment card vs. a playing card). For example, two different payment data sources might use the field names `acct_num` and `PrimaryAccountNumber` for the same concept; a naive embedding-based integration could treat these as separate or even erroneously merge them with other fields named “AccountNumber” from unrelated contexts. Without a robust schema governance mechanism, such misalignments can propagate to downstream analytics or ML models.

Classical normalization (1NF–BCNF) addresses structural redundancy in relational schema design Codd (1970) but not the new semantic challenges above. We propose **Semantic Data Normal Forms (SDNF)** to extend relational normalization into the world of vector-based data semantics. SDNF defines constraints on an SRS (Semantic Relational Schema) that mitigate embedding-related anomalies (instability, drift, aliasing, cross-context confusion, insufficient evidence, etc.). By enforcing these constraints via an evidence-driven **schema governance pipeline** that integrates new data incrementally, our approach yields stable, canonical schemas for machine learning systems, with full lineage and explainability of each integration decision.

Our contributions are:

- (a) A formal **Semantic Relational Schema (SRS) model** with multi-level embeddings and context projection operators, capturing both symbolic and continuous semantic schema information.

- (b) Seven **Semantic Data Normal Forms** (EENF, AANF, CMNF, DBNF, ECNF, RRNF, PONF) to detect and remediate semantic anomalies (instability, aliasing, context mixing, drift, incomplete evidence, role misuse, and partition overlap).
- (c) **Conditional theoretical results** under explicit assumptions, including a cross-context semantic safety guarantee and a convergence argument for schema stability.
- (d) An **evidence-driven governance pipeline** implementing these normal forms to incrementally consolidate schema attributes and maintain semantic integrity.
- (e) **Empirical validation** on a payments-domain dataset, showing substantial schema consolidation (**38.7%** reduction in attribute count), high alias merge precision ($\sim$ **95%**), and minimal context leakage ($\sim$ **2%**) using SDNF, compared to a baseline approach.

2 Semantic Relational Schema (SRS) Model

2.1 Formal definition

We define a **Semantic Relational Schema (SRS)** as a tuple $SRS = (E, A, R, Emb, C, L)$, where:

- E is the set of entities (canonical concepts).
- A is the set of attributes, each with associated metadata (type, regex pattern, provenance, alias history, etc).
- R is the set of semantic relations between attributes and entities (capturing source–target mappings, confidence scores, roles, etc).
- $Emb(p, c)$ is a function mapping a primitive p (e.g., a field name token) in context c to a vector in $\mathbb{R}^d$.
- C is the set of semantic contexts or domains (e.g., “Payments,” “Healthcare”).
- L is an append-only *lineage log* recording changes to the schema over time, including evidence for each change.

Example SRS: Consider a simple SRS for a payments domain. We might have an entity ‘Account’ $\in E$ that represents a user account or card holder, and two attributes in A : `acct_num` and `PrimaryAccountNumber`, which originate from different payment systems but refer to the same concept (account number). A semantic relation in R would link these attributes to the ‘Account’ entity (with a confidence score and evidence pointers) to capture that they are synonyms. This illustrates how the SRS represents distinct source-specific fields as a single canonical concept with traceable provenance in L .

2.2 Multi-level embedding decomposition

For a primitive p in context c , our embedding model produces:

$$Emb(p, c) = [Emb_{fine}(p, c) \parallel Emb_{abstract}(p) \parallel Emb_{contextual}(p, c)] . \quad (1)$$

Each subvector is ℓ_2 -normalized before concatenation. This decomposition separates lexical, context-independent, and domain-specific semantics, supporting progressive convergence as evidence accumulates.

Fine-level embedding. $Emb_{fine}(p, c)$ captures the lexical or token-level semantics of the primitive p within context c .

Explanation: This corresponds to the precise word-level meaning, sensitive to spelling, morphology, and immediate context around p .

Abstract embedding. $Emb_{abstract}(p)$ represents the context-independent, general semantics of the primitive p .

Explanation: This is the distilled, high-level meaning of the word or concept, invariant across different domains (e.g., “card” as a generic concept).

Contextual embedding. $\text{Emb}_{\text{contextual}}(p, c)$ encodes the domain-modulated semantics of the primitive p under context c .

Explanation: This adjusts the meaning of p depending on the domain — for example, the term “operation” in a **Banking** context is understood as a financial transaction, whereas in a **Medical** context it denotes a surgical procedure.

Combining these components allows each attribute’s embedding to be partially decomposable into fine, abstract, and contextual parts. This encourages separation of concerns (by isolating context-specific meaning in the contextual part) and reduces semantic overlap across domains.

2.3 Context projection operator

For each context $c \in C$, we define a linear operator $\text{Proj}_c : \mathbb{R}^d \rightarrow \mathbb{R}^d$ with the following properties:

$$\|\text{Proj}_c(x)\| \leq \kappa, \quad \|\text{Proj}_c^2 - \text{Proj}_c\| \leq \delta, \quad \|\text{Proj}_c(x + \eta) - \text{Proj}_c(x)\| \leq \kappa \|\eta\|. \quad (2)$$

These conditions require that Proj_c provides ****bounded amplification****, ****approximate idempotence****, and ****stability under small perturbations****, respectively.

Intuition: The projection Proj_c acts as a context-specific filter. It never amplifies any component of a vector beyond a factor κ . Applying it twice has nearly the same effect as once (like putting on tinted glasses twice doesn’t change the tint further), and small changes in the input yield only small changes in the output. Practical realizations include PCA-based linear maps and attention-based masks. These projections encourage near-orthogonality of identical tokens across distinct contexts, ensuring that meanings remain separated across domains.

2.4 SRS metric and evidence inner product

Definition 1 (SRS distance). For primitives (p_1, c) and (p_2, c) in the same context c , we define:

$$d_{\text{SRS}}((p_1, c), (p_2, c)) = \|\text{Proj}_c(\text{Emb}(p_1, c)) - \text{Proj}_c(\text{Emb}(p_2, c))\|_2. \quad (3)$$

Explanation: The SRS distance measures how far apart two primitives are in meaning, *after* filtering out their context-specific aspects. If the distance is small, they carry similar meaning within that domain; if large, they diverge semantically in that domain.

Definition 2 (Evidence-weighted inner product). For two entities e_1, e_2 with evidence sets Ev_1, Ev_2 , we define:

$$\langle e_1, e_2 \rangle_E := \sum_i w_i (\text{Ev}_i) \cdot \langle \text{Emb}(e_1), \text{Emb}(e_2) \rangle, \quad (4)$$

where the w_i are normalized nonnegative weights.

Explanation: This defines a similarity measure between entities that accounts for supporting evidence. Think of each piece of evidence as a “vote” with a certain weight; the more or stronger evidence linking e_1 and e_2 , the higher their evidence-weighted similarity. This ensures that similarity is not judged solely by embeddings, but by the strength of evidence behind the semantic connection. For finite evidence sets, this weighted inner product satisfies the properties of an inner product (proof omitted for brevity).

In summary, an SRS provides a structured representation of semantic schemas: d_{SRS} quantifies semantic distance between attributes in a given context, and $\langle \cdot, \cdot \rangle_E$ provides an evidence-aware notion of similarity between entities. We now define normal forms to enforce integrity on the SRS.

3 Semantic Data Normal Forms (SDNF)

Each normal form (NF) addresses a specific type of semantic anomaly in embedding-based schemas. We describe each ****SDNF**** with its constraint, an explanation (including a brief example when relevant), and the remediation triggered if the constraint is violated.

3.1 EENF — Entity Embedding Normal Form (Stability)

Constraint: For each entity e in context c , the embedding variation under repeated encoding must be bounded:

$$\text{Var}(\{\text{Emb}^{(g)}(e, c)\}_{g=1}^G) \leq \tau_{\text{EENF}} , \quad (5)$$

where $\text{Emb}^{(g)}(e, c)$ denotes the g -th independent embedding of e in context c by a nondeterministic model, and G is a fixed number of regeneration trials.

Explanation: The variance across multiple regenerated embeddings of the same entity should remain below a stability threshold τ_{EENF} . If the embeddings fluctuate too widely (indicating high uncertainty in representation), the concept’s meaning is not stable and is at risk of corrupting downstream semantics. For example, if the field “AccountType” yields highly inconsistent embeddings across encodings, it violates EENF.

Action: The system should *quarantine* such unstable attributes and either increase G (the number of encoding trials) or use a more deterministic encoding approach to enforce stability.

3.2 AANF — Attribute Alias Normal Form (Alias Resolution)

Constraint: Two attributes a, b (possibly from different contexts) are flagged as aliases of the same concept if:

$$\begin{aligned} \cos(\text{Emb}(a, c_a), \text{Emb}(b, c_b)) &\geq \tau_{\text{AANF}}, \\ \text{ontology_root}(a) &= \text{ontology_root}(b), \\ \text{EvidenceSet}(a, b) &\geq m_{\min} . \end{aligned} \quad (6)$$

Explanation: Two attributes are identified as potential synonyms (aliases) of the same concept when: (1) their embeddings have high cosine similarity (above τ_{AANF}); (2) they share the same ontology root (i.e., belong to the same conceptual category); and (3) there is sufficient supporting evidence (at least $m_{\min}$ independent signals).

Example: If the dataset contains attributes labeled “DOB” and “Date of Birth,” they could be identified as aliases — they have extremely similar embeddings, match on the same concept in an ontology (both refer to a birth date attribute), and appear in similar contexts.

Action: Automatically merge a and b into one canonical attribute if the above conditions are satisfied (and evidence thresholds per ECNF are met). Otherwise, mark them as aliases but defer the actual merge until more evidence is available.

3.3 CMNF — Context Modulation Normal Form (Context Isolation)

Constraint: For the same primitive p appearing in two contexts c_1 and c_2 ,

$$\left\langle \text{Proj}_{c_1}(\text{Emb}(p, c_1)), \text{Proj}_{c_2}(\text{Emb}(p, c_2)) \right\rangle \leq \tau_{\text{CMNF}} . \quad (7)$$

Explanation: The inner product (cosine similarity) between the context-projected embeddings of the same primitive in two distinct contexts must remain below threshold τ_{CMNF} . This prevents unintentional merging of context-specific meanings of a word.

Example: The term “card” in a **Banking** context (meaning credit card) vs. in a **Gaming** context (meaning playing card) should have a small similarity; if their projected embeddings had high similarity, it would indicate a context overlap problem.

Action: Retrain or adjust the projection operators for those contexts to reduce vector overlap, or flag the overlapping primitives for manual review in case of potential context contamination.

3.4 DBNF — Drift-Bounded Normal Form (Drift Control)

Constraint: For an entity e across two model versions v_1 and v_2 ,

$$\|\text{Emb}(e; v_1) - \text{Emb}(e; v_2)\| \leq \tau_{\text{DBNF}} . \quad (8)$$

Explanation: The distance between the embedding of an entity (or attribute) under two different embedding model versions must remain below drift threshold τ_{DBNF} . This ensures that updates to the embedding model (or significant new training data) do not change the representations of existing concepts so dramatically that their semantic identity is lost.

Example: Updating an embedding model should not cause the concept “bank” (meaning financial institution) to suddenly shift so much in vector space that it appears closer to unrelated concepts. Large drift beyond τ_{DBNF} would indicate that the concept’s representation has shifted unacceptably.

Action: If drift surpasses the threshold, the pipeline *forks* the schema or model for e (preserving the old representation separately) to maintain historical consistency. This prevents silent corruption of meaning by enabling side-by-side existence of distinct interpretations for e .

3.5 ECNF — Evidence Completeness Normal Form (Explainability)

Constraint: A schema operation O (like merging two attributes) is permitted only if it has sufficient supporting evidence:

$$\text{EvidenceSet}(O) \geq m_{\min}, \quad \text{aggregate_score}(O) \geq \gamma. \quad (9)$$

Here $m_{\min}$ is a minimum count of distinct evidence signals (e.g., name similarity, ontology, etc.), and γ is a minimum aggregate confidence score.

Action: If the evidence criteria are satisfied, perform the operation automatically (e.g., merge aliases); otherwise, postpone the operation for human review or until more evidence is gathered.

Practical note: In our data, value-based evidence (comparing actual field values) is often missing (27.6% of attributes). The pipeline compensates by combining alternative evidence signals (name, ontology, VSS, shape, embeddings) in a hybrid mode (see Appendix C), which preserves high precision while improving recall.

In our implementation, the $\text{aggregate_score}(O)$ is computed as a weighted sum of available evidence signals:

$$\text{aggregate_score}(O) = \sum_{s \in S} w_s \text{score}_s(O), \quad \sum_{s \in S} w_s = 1, \quad (10)$$

where S is the set of evidence signal types (embedding similarity, name overlap, ontology tag match, value shape similarity, value semantic signature, etc.), and $\text{score}_s(O) \in [0, 1]$ is the normalized score provided by signal s for operation O . We assign larger w_s to more reliable signals; for instance, in our experiments we set $w_{\text{emb}} = 0.4$, $w_{\text{name}} = 0.2$, $w_{\text{ont}} = 0.1$, $w_{\text{shape}} = 0.1$, $w_{\text{VSS}} = 0.2$, reflecting greater trust in embedding-based and value-distribution signals. These weights were initially tuned on a small calibration set (to maximize alias detection F_1), and moderate ± 0.05 adjustments to individual w_s did not significantly affect precision or recall.

3.6 RRNF — Role-Respecting Normal Form (Role Consistency)

Constraint: Each entity or attribute plays a consistent *role* in semantic relations; it cannot change roles across different relations.

Explanation: This ensures that relationships maintain logical consistency.

Example: In a family tree context, an entity labeled as a “parent” should not appear as a “child” in a different relationship. Such a role inversion would violate RRNF.

Action: If an entity or attribute’s role is found to be inconsistent across relations, the inconsistent relation is flagged, and manual correction or re-annotation is required to maintain semantic coherence.

3.7 PONF — Partition Orthogonality Normal Form (Semantic Separation)

Constraint: Maintain orthogonality of semantic partitions. For any two partitions P_i and P_j of entities or attributes in the SRS,

$$\text{Overlap}(P_i, P_j) \leq \tau_{\text{PONF}}, \quad \forall i \neq j, \quad (11)$$

meaning the overlap between any two distinct semantic clusters remains below the threshold τ_{PONF} .

Explanation: This ensures that distinct conceptual partitions (e.g., separate ontology branches or clusters

of entities) remain well-separated in semantic space. If two clusters begin to overlap significantly (exceeding τ_{PONF}), it indicates a taxonomy or clustering problem.

Action: Monitor the semantic clustering of the SRS; if partitions are found to overlap, adjust the clustering or increase embedding dimensionality to restore orthogonality.

4 Mathematical Analysis

4.1 Assumptions

To formalize SDNF, we state explicit assumptions under which theoretical guarantees can be derived. All assumptions are primarily to enable analysis and can be relaxed in practice.

(A1) Projection stability. Each context projection Proj_c is *bounded, approximately idempotent, and stable* as defined in Section 2.3.

Boundedness: Projections never amplify signals beyond factor κ .

Approx. idempotent: $\text{Proj}_c(\text{Proj}_c(x))$ is essentially the same as $\text{Proj}_c(x)$ (within δ).

Stability: Small changes in input cause proportionally small changes in output.

(A2) Embedding Lipschitz. The embedding function Emb is Lipschitz continuous on the finite vocabulary in each context, and approximately injective on that domain.

Intuition: Similar meaning differences produce limited changes in embeddings, and distinct primitives do not collapse to identical embeddings.

(A3) Subspace model. Embeddings for different contexts tend to lie in distinct low-dimensional subspaces of $\mathbb{R}^d$ (as encouraged by the Proj operators).

Intuition: We can think of each context as occupying its own subspace or “semantic subroom” in the high-dimensional space, with limited overlap as formalized by concentration bounds.

(A4) Governance descent. The SDNF governance pipeline steps (especially merges and forks) are designed such that a redundancy metric $L(S)$ on the current schema state is non-increasing in expectation at each step.

Intuition: Each operation reduces some measure of redundancy or potential error in the schema, ensuring progress toward a cleaner schema without introducing new inconsistencies.

These assumptions allow us to state our theorems precisely; detailed proofs are provided in Appendix A–B.

4.2 Conditional theorems (statements and sketches)

Under assumptions A1–A4, we establish three theoretical results. We provide statements and intuitive explanations here, deferring formal proofs to appendices.

Theorem 1 (Cross-Context Safety under CMNF). Suppose c_1 and c_2 are distinct contexts satisfying the CMNF constraint (i.e., for all primitives p , $\langle \text{Proj}_{c_1}(\text{Emb}(p, c_1)), \text{Proj}_{c_2}(\text{Emb}(p, c_2)) \rangle \leq \tau_{\text{CMNF}}$). Then, under assumptions A1–A4 (especially the subspace model), the probability of a cross-context retrieval error is bounded by a function $f(\tau_{\text{CMNF}}; d, n)$ that decreases towards 0 as $\tau_{\text{CMNF}} \rightarrow 0$ and the embedding dimension d increases. (*Proof sketch:* If contexts are nearly orthogonal and embedding differences concentrate in smaller subspaces, then mistaking an entity from c_1 for one in c_2 becomes exponentially unlikely in d . See Appendix A.)

Theorem 2 (Drift Fork Necessity under DBNF). Under A1–A4, if an entity e ’s representation across two model versions has distance $\|\text{Emb}(e; v_1) - \text{Emb}(e; v_2)\| > \tau_{\text{DBNF}}$, then for any L_O -Lipschitz continuous downstream operator O , the change in O ’s output on e between the two versions is lower-bounded by $L_O \cdot (\text{drift magnitude})$ (minus a constant). If this output change exceeds a safe threshold, a schema fork is necessary to preserve consistency. (*Proof idea:* Strong drift implies a significant semantic shift, which a Lipschitz operator will propagate to outputs in proportion. Beyond a threshold, continuing with one schema would cause output anomalies; thus a fork is required. See Appendix A.)

Theorem 3 (Schema Convergence under SDNF). Under A1–A4, and for finite contexts with bounded update steps in the pipeline, repeated application of SDNF governance is convergent. In particular, the SRS metric space $(\mathcal{X}, d_{\text{SRS}})$ is complete (every Cauchy sequence converges in the space), and the SDNF governance pipeline updates monotonically reduce the redundancy metric $L(S)$, ensuring eventual convergence to a fixed-point canonical schema S^* (see Corollary 3.1 below, and Appendix B). (*Intuition:* Because each SDNF enforcement step cleans up redundancy and is limited in impact, the sequence of schema states forms a Cauchy sequence in the SRS metric. It converges to a stable “final schema” where no further merges or forks are needed.)

Corollary 3.1 (Fixed-Point Convergence). Under the above conditions, the governance pipeline converges to a final canonical schema S^* that remains unchanged under further governance operations (i.e., it is a fixed point).

Corollary 3.2 (Conditional Bi-Lipschitz Embedding). Under assumption A2 (finite contexts), the embedding function Emb is bi-Lipschitz (with constants depending on κ and τ_{AANF}). (*Intuition:* In a finite domain, if the embedding is Lipschitz and no two distinct primitives share identical embeddings, then the embedding approximately preserves distances: similar objects remain close and distinct ones remain sufficiently apart.)

4.3 Governance descent and convergence

The SDNF governance pipeline can be viewed as a sequence of continuous updates (e.g., re-centering of clusters) interleaved with discrete structural decisions (merges, forks). The continuous steps are Lipschitz-bounded, and the discrete steps are designed to always reduce (or at least not increase) the redundancy metric $L(S)$. Under additional boundedness conditions on the effect of each discrete operation (Appendix B), the pipeline’s repeated application yields a monotonic decrease in $L(S)$ and eventual convergence to a unique fixed point (Banach’s fixed-point theorem).

5 Governance Pipeline

To implement SDNF in practice, we develop a governance pipeline that incrementally ingests new data and enforces semantic normal forms step by step. Below we outline each stage of this pipeline, continuing our running example of consolidating payment card account number fields:

1. **Payload ingestion:** New data payloads (e.g., a JSON record from a payment system) are parsed, and their field names and values are extracted.
2. **Schema derivation:** For each source dataset, we form an initial schema of fields and data types. The system identifies the structural elements and adds new primitives (field labels) to the SRS as candidate attributes.
3. **Embedding generation:** Using a Sentence-BERT model (all-MiniLM-L6-v2 Reimers & Gurevych (2019)), we compute the multi-level embeddings for each new attribute in its context (per Section 2.2). These embeddings are indexed (e.g., via an HNSW library) to support efficient similarity queries.
4. **Context projection:** Each embedding is projected into its domain-specific subspace by the corresponding Proj_c (Sec. 2.3). This filters each attribute’s representation to emphasize domain-relevant dimensions and suppress cross-context interference.
5. **SDNF validation:** The pipeline then checks each new or updated attribute against all seven SDNF constraints (EENF through PONF) in order. If any normal form is violated, the attribute is marked for special handling (e.g., potential merge, drift management, etc.) as determined by that NF.
6. **Evidence aggregation:** To make robust decisions, we aggregate multiple signals about attribute similarity and relatedness, including embedding cosine similarities (across fine/abstract/contextual components), lexical token overlaps, ontology matches, value co-occurrence statistics, regex pattern matches (for value shapes), and value semantic signatures (VSS). We compute a weighted aggregate confidence score from these signals.

7. **Decision (merge/fork/quarantine/defer):** Based on the SDNF checks and evidence score, the pipeline decides how to integrate the attribute: if AANF and ECNF indicate a strong alias, the attribute is **auto-merged** with an existing canonical concept; if DBNF indicates unacceptable drift, a **fork** is initiated; if EENF shows instability, the attribute is **quarantined**; otherwise, the attribute (or detected alias relationship) is **deferred** until more evidence is available.
8. **Lineage logging:** Each operation is recorded in the lineage log L with a timestamp, references to evidence, and a rationale, ensuring auditability of how and why each merge, fork, or other action was taken.

Operational guidelines. In summary, our pipeline only performs a merge when both **AANF** and **ECNF** constraints are satisfied, ensuring each consolidation is evidence-backed; any severe drift triggers a fork per **DBNF**; any unstable embedding triggers a quarantine per **EENF**; and actions are deferred when evidence falls short. These rules make schema integration precise and cautious, while still reducing redundancy over time.

6 Experiments

To evaluate SDNF, we conduct a schema integration experiment on a real-world dataset from the **payments domain**. We compare our SDNF pipeline to a baseline approach and perform ablations to assess the impact of each normal form and evidence type.

6.1 Experimental Setup

Platform: All experiments run in Python 3.10 using the Sentence-Transformers **all-MiniLM-L6-v2** model Reimers & Gurevych (2019) for text embeddings. Nearest neighbor searches are accelerated with an HNSW approximate vector index (via **hnswlib**, parameters $M = 32$, **ef_search**=50) for efficiency.

Data and Task: We compiled a dataset of seven heterogeneous JSON schemas from the payments domain (e.g., credit card and e-commerce transaction formats), along with approximately 50 sample payloads across these sources. The combined schemas contain a total of 80 distinct attributes, many of which are semantic duplicates across sources. The task is to incrementally integrate these sources into a unified SRS while consolidating semantically equivalent attributes and preserving context distinctions.

Metrics: We measure: (i) **Schema Consolidation**, the percentage reduction in distinct attribute count after integration; (ii) **Alias Merge Precision/Recall**, comparing detected merges against a manually curated ground truth mapping of equivalent attributes; (iii) **Cross-Context Leakage**, the fraction of merges that inappropriately combine attributes from different contexts (i.e., context isolation failures); and (iv) **Drift Detection Accuracy**, the fraction of known simulated drift events that are correctly detected by the pipeline. We run experiments with fixed random seeds and provide implementation details to facilitate reproducibility (Appendix C).

6.2 EENF Mitigation and Reproducibility

Embedding model nondeterminism can cause identical inputs to have slightly different embeddings across runs, which threatens EENF. We quantify and mitigate this by using multiple encoding trials (regenerations). Specifically, we set $G = 10$ and observe that this reduces the mean embedding variance by $\sim 40\%$ (vs. single encoding) at the cost of doubling encoding time. Increasing to $G = 20$ further halves the variance (Appendix C shows measured variances and quarantine triggers). All experiments use a fixed global random seed and identical payload ingestion order to ensure deterministic outcomes (Appendix C). For deployment, we recommend either using deterministic embedding models or applying EENF with sufficient G to guarantee stable representations. The overhead of repeated embedding trials grows linearly with G , so we analyze the stability-latency trade-off. In our experiments, $G = 10$ cut variance by $\sim 40\%$ while roughly doubling encoding time (relative to $G = 1$), and $G = 20$ cut variance by $\sim 70\%$ with approximately $4\times$ encoding time.

Table 1 summarizes these trade-offs and provides guidelines for choosing G .

G	Variance Reduction	Encoding Overhead	Recommended Use
1	0% (baseline)	$1\times$	Model very stable or real-time
10	$\sim 40\%$	$\sim 2\times$	Default (balanced)
20	$\sim 70\%$	$\sim 4\times$	Maximum stability required

Table 1: Stability-latency trade-offs for repeated embedding (G regeneration trials). Overhead is measured relative to a single encoding ($G = 1$).

6.3 Key Results

We compared our SDNF pipeline to a baseline approach on the payments integration task. The baseline merges attributes solely based on high embedding cosine similarity (above a fixed threshold), using no normal form constraints or evidence signals. The two methods achieve similar schema consolidation but differ markedly in precision and context isolation. The results are summarized below:

Approach	Schema Reduction	Merge Precision	Merge Recall	X-Context Leakage
SDNF Pipeline (ours)	38.7%	95%	90%	$\sim 2\%$
Baseline (embed-only)	38.7%	86%	95%	$\sim 9\%$

Table 2: Schema integration results on the payments dataset. The baseline merges attributes by embedding similarity threshold only, without normal form constraints. SDNF achieves the same consolidation rate but with substantially higher precision and minimal cross-context errors.

As shown in Table 2, both SDNF and the baseline achieve a consolidation of 38.7% (reducing 80 attributes to 49). However, the baseline’s aggressive merging leads to some incorrect merges (precision $\approx 86\%$) and multiple context-mixing errors (about 9% of merges combined fields from different domains). In contrast, SDNF maintains a high merge precision ($\approx 95\%$) by enforcing ECNF (merging only when sufficient evidence is present) and limits cross-context errors to under 2% thanks to CMNF (context isolation). The recall (fraction of true aliases merged) for SDNF is slightly lower (90% vs. 95% for the baseline) due to its more conservative criteria, but still high.

6.4 Ablation Study

We conducted ablation experiments to examine each SDNF component’s effect:

- **No ECNF:** Without enforcing evidence completeness, alias merge precision drops to $\sim 86\%$. This highlights that requiring multiple evidence signals is essential to avoid false merges.
- **No CMNF:** Without context isolation, cross-context merging errors increase to $\sim 9\%$ (similar to the baseline’s outcome), confirming that CMNF effectively prevents semantic contamination between domains.
- **No DBNF:** Without drift handling, simulated embedding drifts are undetected, eventually causing unchecked semantic corruption of the schema.
- **No value evidence:** In our dataset, 27.6% of attributes lack direct value samples. In these cases, using the hybrid evidence mode (combining name, ontology, VSS, shape, and embedding signals) was crucial to maintain recall without sacrificing precision. When only name or embedding evidence was used, many true alias pairs remained unmerged due to missing values; the hybrid approach recovered most of these merges (Appendix C).

6.5 Illustrative Merge Example

To illustrate the pipeline’s behavior, consider the consolidation of an alias pair from the payments data. The attribute `acct_num` (from a bank dataset) and `PrimaryAccountNumber` (from a credit card schema)

were identified as aliases by our pipeline. Their embeddings had a cosine similarity of 0.99, they shared an ISO 8583 “PAN” ontology tag, their values co-occurred in 82% of records, and their formats both matched the regex pattern $\sim[0-9]\{13,19\}\$$. The aggregated confidence score for merging them was 0.92 (exceeding our threshold γ), so the pipeline automatically merged these two attributes and recorded a detailed lineage entry describing the evidence and operation.

7 Related Work

Classical database normalization theory Codd (1970); Kent (1979) fundamentally addresses structural redundancy in schema design. Our work extends those ideas into the realm of vector semantics and continuous representations, where new challenges of schema integrity arise. The literature on **schema matching** and ontology alignment (cf. the survey by Rahm and Bernstein Rahm & Bernstein (2001)) informs our approach to alias detection via combined signals. Meanwhile, advances in **vector databases and nearest-neighbor retrieval** (e.g., HNSW indexes Malkov & Yashunin (2018) and approximate search on GPUs Johnson et al. (2019)) guide the scalable implementation of our embedding-based matching. Even though we focus on conceptual rather than linguistic semantics, concerns about **representation drift** in evolving embeddings have influenced our DBNF constraint to manage semantic changes across model updates. Finally, our theoretical analysis leverages results in metric space geometry and concentration inequalities Heinonen (2001); Boucheron et al. (2013) to provide safety guarantees, and draws on fixed-point theory Banach (1922) to reason about the convergence of the schema refinement process.

8 Conclusion

We presented SDNF, a practical framework for governing semantic schemas in AI/ML data pipelines. By combining a formal SRS model, seven semantic normal forms, and an evidence-driven governance pipeline with lineage, SDNF helps reduce schema redundancy and mitigate semantic anomalies (like instability, drift, and context mixing). Our theoretical results provide conditional assurances of cross-context safety and eventual schema convergence under manageable assumptions, and our empirical study on payments data demonstrated significant schema consolidation and high alias merge precision with SDNF.

Future work will generalize SDNF to other domains (e.g., non-financial datasets), explore **scalable deployment** (e.g., hierarchical partitioning and indexing for enterprise-scale schemas), and aim to relax the theoretical assumptions for broader applicability.

Limitations

Domain scope: Our experimental evaluation is limited to payments data; further studies are needed to confirm that SDNF performs well in other domains and on larger-scale schemas.

Nondeterminism: SDNF addresses encoder nondeterminism (stochastic embeddings) via regeneration and seeding; for highly unstable models, additional measures might be required to satisfy EENF.

Evidence gaps: Some attributes lack value samples, limiting direct evidence for ECNF. Our hybrid evidence strategy mitigates this, but it relies on domain-specific signals (like ontology tags) that may not always be available.

Scalability: Our prototype and experiments are modest in scale; more work is needed to ensure scalability to enterprise-scale schemas. *Repeatedly encoding each attribute G times for EENF enforcement introduces overhead linear in G . With millions of attributes, this becomes a significant latency bottleneck. Mitigation strategies include caching stable embeddings, using partially deterministic encoders, or dynamically increasing G only for high-variance cases. Another factor is similarity search: we leverage an HNSW approximate nearest neighbor index Malkov & Yashunin (2018) to reduce retrieval complexity (from $O(N)$ brute-force to roughly $O(\log N)$ in practice). A promising direction is to tailor HNSW to SDNF’s context structure: for example, building separate HNSW graphs per context (domain) or adding context labels to nodes to restrict cross-context neighbor connections. This could further cut candidate comparisons while preserving context isolation (CMNF) and alias resolution semantics (AANF). As future work, designing context-partitioned*

and projection-aware HNSW indexing might achieve near-linear scaling while maintaining SDNF’s semantic integrity.

Reproducibility: We include details of model versions, seeds, and library configurations (Appendix C), and we provide an anonymous code submission to ensure all results are reproducible.

References

- Stefan Banach. Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae*, 3:133–181, 1922.
- Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013. doi: 10.1093/acprof:oso/9780199535255.001.0001.
- E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6): 377–387, 1970. doi: 10.1145/362384.362685.
- Juha Heinonen. *Lectures on Analysis on Metric Spaces*. Springer, 2001. doi: 10.1007/978-1-4471-3689-1.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. doi: 10.1109/TBDATA.2019.2921572.
- William Kent. *Data and Reality: Basic Assumptions in Data Processing Reconsidered*. North-Holland, 1979.
- Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4):824–836, 2018. doi: 10.1109/TPAMI.2018.2889473.
- Erhard Rahm and Philip A. Bernstein. A survey of approaches to automatic schema matching. *VLDB Journal*, 10(4):334–350, 2001. doi: 10.1007/s007780100057.
- Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3982–3992, 2019. URL <https://arxiv.org/abs/1908.10084>.

A Reproducibility & Implementation Details

To aid reproducibility, we include additional details about our experiments. All code and results are provided with the submission, and will be openly released upon acceptance.

A.1 Model, Data, and Hyperparameters

We use the Sentence-Transformers `all-MiniLM-L6-v2` model (384-dimensional embeddings). To simulate drift (for DBNF evaluation), we also use a second embedding model (`all-mpnet-base-v2`, 768-dimensional) for a subset of attributes. The seven source schemas consist of JSON files from payment processing APIs (e.g., Visa, MasterCard, ISO 20022) along with their sample payloads. They are integrated sequentially, and initial thresholds are calibrated on a held-out subset as described below. All experiments were run on a standard laptop with a 2.8 GHz CPU (processing time was under 5 minutes for the entire pipeline).

A.2 Threshold Calibration

SDNF thresholds were determined empirically via a small calibration set:

- τ_{EENF} : 95th percentile of per-attribute embedding variance across $G = 10$ regenerations.
- τ_{AANF} : threshold mid-way between distribution of similarities for known alias pairs vs. random pairs.

- τ_{CMNF} : chosen based on cross-context projection overlap statistics after initial PCA orthogonalization.
- τ_{DBNF} : set from high quantiles of embedding shifts observed when comparing the primary embedding model to a prior version.

For our experiments, the calibrated values were: $\tau_{\text{EENF}} = 1.29 \times 10^{-4}$, $\tau_{\text{AANF}} = 0.65$, $\tau_{\text{CMNF}} = 0.10$, $\tau_{\text{DBNF}} = 0.25$.

A.3 Additional Experimental Results

Our implementation logs all key outcomes in a machine-readable format (see provided output file). For example, the final integrated schema had 49 attributes; the pipeline’s lineage log recorded each merge decision with references to supporting evidence. We also measured that each merge decision took under 50 ms on average, indicating the pipeline’s efficiency. For further details, including extended logs and full pseudocode of the pipeline, see the anonymized code repository (provided as supplemental material).

A.4 Alternative Evidence Signals and Weighting Scheme

To complement Section 3.5, we describe the evidence signals and weighing scheme used for the aggregate confidence score. We consider five evidence signals for a candidate merge: (i) *embedding similarity* (cos distance between attribute vectors), (ii) *name similarity* (lexical token overlap or edit distance between attribute names), (iii) *ontology tag match* (binary indicator if attributes share a taxonomy or ontology category), (iv) *value shape similarity* (pattern or regex-based feature for value formats), and (v) *value semantic signature (VSS)* which encodes an attribute’s value distribution or sample. Each signal s yields a normalized score $\text{score}_s \in [0, 1]$. The aggregate confidence score for merging two attributes a, b is then:

$$\begin{aligned} \text{score}(a, b) = & w_{\text{emb}} \cdot \cos(\text{Emb}(a), \text{Emb}(b)) + w_{\text{name}} \cdot \text{sim}_{\text{name}}(a, b) \\ & + w_{\text{ont}} \cdot \mathbb{I}[\text{OntRoot}(a) = \text{OntRoot}(b)] + w_{\text{shape}} \cdot \text{sim}_{\text{shape}}(a, b) \\ & + w_{\text{VSS}} \cdot \text{sim}_{\text{VSS}}(a, b). \end{aligned} \quad (12)$$

with $\sum w = 1$. We used default weights $w_{\text{emb}} = 0.4$, $w_{\text{name}} = 0.2$, $w_{\text{ont}} = 0.1$, $w_{\text{shape}} = 0.1$, $w_{\text{VSS}} = 0.2$, giving highest weight to embedding and moderate to value-distribution signals. If a particular evidence type is unavailable (e.g., no value samples, hence no VSS), its weight is redistributed proportionally among the remaining signals so the sum of weights remains 1. We found the alias merge results to be robust to moderate variations in these weights (e.g., altering each w_s by ± 0.05 yielded $\leq 1\%$ change in F_1). In future work, these weights could be learned (e.g., via logistic regression on labeled merges), but we opted for a fixed scheme given limited ground-truth data.

A.5 SDNF-Aware Indexing Strategies at Scale

For completeness, we outline potential HNSW-based indexing customizations to support SDNF at scale:

- **Context-partitioned HNSW Graphs:** Maintain separate HNSW graphs for each semantic context (domain). This limits neighbor exploration to relevant contexts by design, drastically reducing comparisons for common case queries. Cross-context alias detection can still be performed by occasionally querying multiple indices (with careful evidence gating to preserve CMNF).
- **Projection-aware Distance:** Pre-apply Proj_c on embeddings before indexing, or incorporate context filters at query time. This ensures HNSW’s similarity metric respects context isolation—neighbors are ranked by their projected distance in the appropriate subspace, preventing context mixing while refining search space.
- **Hybrid ANN + SDNF Validation:** Use HNSW to retrieve a short-list of top- k candidate matches (sublinear time retrieval), and then apply precise SDNF constraint checks (AANF, ECNF, etc.) on those

candidates to finalize merges. This approach combines the speed of approximate search with the guarantee that only SDNF-compliant merges are executed.

Indexing Strategy	Scalability Benefits	SDNF Compliance
Global HNSW with query-time context filter	Simple design; catches cross-domain similarity	Full compliance; filter ensures CMNF
Context-partitioned HNSW indices	Near-linear scaling (each index smaller); context isolation	Guaranteed CMNF; cross-context merges via specialized queries
Context-labeled HNSW edges	Single graph, but restrict cross-context links	Partial compliance; risk of missed cross aliases
Hybrid (HNSW retrieval + SDNF checks)	Good speed-up; easy to implement	Full compliance (SDNF enforced in post-processing)

Table 3: Potential HNSW-based indexing strategies for scaling SDNF, with their benefits and how they interact with SDNF constraints.